INTERVIEW STUDY GUIDE · VOLUME 2

Firmware and Platform Deep Dive — Gently

A beginner-first explanation of what happens when the PC sends one AI input to the AM13E230 board, why every part exists, and only then how the code implements it.

AM13E230 TINIElite Edge AI · Jainil Bavishi · November 2026

How to use this volume

You do **not** need to memorize every register, address, macro, or symbol in this document. First understand the story in Sections 1–3. Then read the later sections when you want to know how that part is implemented.

1. The whole project in one simple story
2. Six words that unlock the rest
3. Follow one inference from PC to result
4. How source files become board firmware
5. Three configuration systems, three jobs
6. Why clock setup matters
7. How the PC and board talk
8. How the compiled model is connected
9. How the NPU and interrupt work together
10. What happens when something goes wrong
11. Where to look in the repository
12. What to say in an interview

1. The whole project in one simple story

Imagine a small AI laboratory split into two parts:

- **The PC is the test supervisor.** It prepares an input, sends it to the board, receives the result, and checks whether the result is what the supplied model expects.
- **The AM13E230 board is the worker.** It runs the AI model locally. It does not send the input to a cloud service.

```
PC: prepare one test tensor
        |  USB UART
        ▼
AM13E230 board: validate the request and start the model
        |
        ▼
Generated AI runtime: CPU work + supported NPU stages
        |  output scores + timing + interrupt counters
        ▼
PC: check the result and diagnostics
```

The difficult part is not simply calling an AI function. The board is a preview device with no ready-made upstream Zephyr support. The project supplies the missing platform layer: board description, startup clock setup, UART, pins, watchdog, TINIElite NPU driver, test protocol, and build process.

> **THE MAIN IDEA TO REMEMBER**
> We took a supplied compiled AI model and built a trustworthy embedded system around it so it can run on the board and be tested repeatedly.

2. Six words that unlock the rest

Firmware

The program stored in a microcontroller's flash memory. Here, `zephyr.hex` is programmed onto the AM13E230.

Driver

Code that knows how to control one hardware block. The UART driver sends bytes; the TINIElite driver controls the NPU.

Register

A tiny hardware-controlled memory location used to configure or observe a peripheral. A UART register holds received data; an NPU register reports completion.

Interrupt

A hardware notification that briefly gets the CPU's attention. The NPU raises DONE interrupt 41 when one accelerator stage finishes.

Semaphore

A software signal that makes a task wait until an event happens. The application waits on one instead of constantly checking the NPU.

ABI

An exact agreement between already-compiled software pieces. The supplied model archive expects certain function names and data layouts.

One useful analogy: the NPU is a specialist worker, an interrupt is its “I am done” bell, and a semaphore is the application waiting by the bell rather than repeatedly asking “are you done yet?”

3. Follow one inference from PC to result

This is the most important section. If you can explain this sequence in your own words, you understand the centre of the project.

1. **Build selection.** Before flashing, the developer selects a model package, such as `waveform_classifier` or `electrical_fault_6class`.
2. **Board boots.** Firmware sets up the clock, pins, UART, NPU, watchdog, and application.
3. **Board says READY.** It sends its model identity, expected input size, output size, and data types to the PC.
4. **PC checks the contract.** The PC refuses to send an input if it selected a different model or tensor shape. This prevents silent nonsense, such as sending 256 bytes to a model that expects 3,072 bytes.
5. **PC prepares input.** For the waveform model, it converts 256 floating-point samples into the required signed 8-bit values. For the electrical-fault model, it sends 768 float values in the declared channel order.
6. **PC sends a protected message.** The UART frame includes a message type, sequence number, length, and CRC error check.
7. **Firmware validates it.** The board checks that the message is complete, the size is correct, and the CRC matches before starting AI work.
8. **The generated model begins.** A small generated adapter passes the input/output buffers to the correct generated model function.
9. **CPU and NPU cooperate.** The generated runtime performs some work on the Cortex-M33 CPU and gives supported stages to TINIElite.
10. **The NPU rings the bell.** Each finished hardware stage raises the DONE interrupt. The short interrupt handler records it and wakes the waiting application.

11. **The model continues or finishes.** The normal application context, not the interrupt handler, calls the generated continuation code. Eventually the output scores are ready.
12. **The board replies.** It returns output bytes, timing values, and interrupt counters. The PC checks output and diagnostics against the package fixtures.

> **IMPORTANT SCOPE**
> Exact supplied fixtures prove that this integrated binary behaves deterministically for those cases. They do not prove broad real-world model accuracy.

4. How source files become board firmware

4.1 Start with a simple picture

```
Project C files + Zephyr C files + supplied model archive
                          |
                          ▼
                      compiler
                          |
                          ▼
                object files (.o files)
                          |
                          ▼
                       linker
                          |
                          ▼
              zephyr.elf  →  zephyr.hex  →  board flash
```

The compiler turns human-written C into processor instructions. Each C file becomes an **object file**, usually ending in `.o`. An object file can refer to functions stored elsewhere; it does not need to be a complete program on its own.

The **linker** is the final assembler. It joins Zephyr, this project's drivers, the application, and the model archive. It resolves function names and chooses final flash/RAM addresses. The result is `zephyr.elf`, a complete firmware executable. A conversion step creates `zephyr.hex`, which the XDS110/DSLite tool programs to board flash.

4.2 What is `mod.a`?

`mod.a` is a **static library**, also called an archive. Think of it as a box containing already-compiled object files. TI's model-generation tool created it. It contains generated model/runtime code and NPU-related code; it is not the original trainable model source.

That is why this is accurately called an **integration project**. The work was to supply the environment that the compiled archive needs, not to claim the neural network was trained from scratch in this repository.

4.3 Why the linker check matters

Putting an archive in a folder does not prove its code reached the board. The build forces the model entry function to be linked, and checks the final ELF for model, NPU, and project symbols. This is similar to checking

that an ingredient was actually put into a cake, not merely left in the kitchen.

5. Three configuration systems, three jobs

These names are intimidating only because they all run during a build. They do different jobs:

System	Simple question	Example in this project
CMake	What code and libraries should be built?	Compile application/driver C files; link the selected `mod.a`.
Kconfig	Which software options should exist in this firmware?	Enable drivers, set stack size, optionally enable diagnostic fault tests.
Devicetree	What physical hardware exists and how is it wired?	UART address, NPU IRQ 41, pin assignment, watchdog and clock hardware.

5.1 Kconfig: software choices before compiling

Kconfig options begin with `CONFIG_`. For example, a diagnostic option can include deliberately dangerous test messages that simulate an NPU stall or watchdog failure. Those options are chosen while building; they are not normal buttons the PC can turn on after the board is flashed.

5.2 Devicetree: the board's hardware map

Devicetree describes hardware facts: where a peripheral's registers are, which interrupt it uses, and which pins connect to it. The UART does not “move” because C code wants it to; its address and pins are physical properties. The driver reads those facts from devicetree-generated configuration.

5.3 Why this repository has many top-level platform folders

Zephyr discovers boards, SoCs, devicetree files, drivers, and headers at specific module roots. Therefore `boards/`, `soc/`, `dts/`, `drivers/`, and `include/` correctly stay at the repository root. They are not random clutter. Moving them under a cosmetic folder would break Zephyr's standard discovery architecture.

5.4 What “out of tree” means in this project

The project does not edit the downloaded upstream Zephyr source tree. Instead, `zephyr/module.yml` tells Zephyr to also search this repository for board, SoC, devicetree, CMake, and Kconfig content. That is called **out-of-tree support**: the platform participates in the build without becoming a hidden local modification of upstream Zephyr.

6. Why clock setup matters

A clock is the CPU's beat. Faster beats mean the CPU can execute more steps per second, but other hardware such as flash and UART must be configured to match. This board is intended to run at 200 MHz.

> **A REAL BUG THAT TAUGHT AN IMPORTANT LESSON**
> Software once *claimed* the CPU ran at 200 MHz, but hardware was still running at 32 MHz. Timing calculations based on 200 MHz were therefore wrong. A number in a configuration file is only a promise; registers must be set and read back to prove the hardware agrees.

6.1 What the boot code does

1. Set flash wait states. At high CPU speed, flash needs extra clock cycles to safely provide instructions.
2. Start the board's 25 MHz crystal oscillator.
3. Configure and lock the PLL. A PLL is hardware that derives a faster, stable clock from a reference clock.
4. Use the TI sequence to produce 200 MHz: `(25 MHz / 2 × 32) / 2 = 200 MHz`.
5. Switch to that clock in safe stages rather than abruptly.
6. Read back raw registers and report whether the expected configuration is genuinely active.

The few instructions that change flash timing run from RAM, not flash. This avoids changing the timing rules of the storage currently supplying the instructions being executed.

6.2 Why the UART cares about the clock too

UART baud rate is derived from its clock. If the driver believes it has a 100 MHz UART source but hardware supplies something else, “115200 baud” is not really 115200 baud and the PC may see corrupt text. The project therefore checks both MCLK and UART-source assumptions.

7. How the PC and board talk

7.1 Pins are not automatically UART pins

A microcontroller pin can usually perform several jobs. PA0 and PA1 must be routed to the UART peripheral before bytes can leave or enter the board. This is called **pin control**, or pinctrl. Without it, the UART may be configured internally but electrically disconnected from the outside world.

7.2 UART is deliberately simple here

The driver uses polling for individual UART bytes: it checks whether receive data exists or transmit space is available. That is simple and useful during board bring-up. It is independent of the NPU design: UART can poll while NPU completion uses an interrupt.

Polling UART is acceptable for this single-request test server, but an interrupt-driven or asynchronous buffered UART is a future improvement for throughput-heavy use.

7.3 The protective message envelope

Raw UART is just a stream of bytes. The receiver needs to know where one message begins, how long it is, and whether it was damaged. Protocol v3 wraps every message in this envelope:

Part	Plain purpose
Magic bytes	Special starting pattern used to find the next message.
Version	Stops two incompatible protocol versions from pretending to understand each other.
Message type	Says whether this is input, result, READY, error, or status.
Sequence number	Connects a reply to the correct request.
Length	Tells the receiver how many payload bytes must arrive.
CRC-32	Mathematical fingerprint used to detect accidental corruption.

The firmware validates the frame before using it. If bytes are damaged, it returns a structured error and looks for the next valid magic pattern rather than treating corrupted bytes as an AI input.

7.4 READY versus ACK

`READY` means: “I have booted, I am idle, and here is the exact model/tensor contract linked into me.” `ACK` means: “I accepted this particular numbered request and will now work on it.” Keeping these separate makes diagnosis much clearer.

8. How the compiled model is connected

8.1 Why one common application cannot directly call every model

Generated model packages may use different input sizes, data types, C struct names, struct field names, and output sizes. If the common UART/NPU code directly named these details, every new compatible model would require editing many unrelated files. That would be fragile.

8.2 The model manifest is the label on the package

Each `models/<name>/model.json` declares facts such as:

- where the archive and generated header live;
- input/output shape, size, type, labels, and alignment;
- how host-side preprocessing must work;
- the generated C struct and field names; and
- the generated entry, completion, and continuation symbols.

8.3 The generated adapter: the small translator

```
model.json says:
  input field name = input_1
  output field name = output
  entry function = tvmgen_default_run

generator writes build/generated/model_adapter.c:
  app_model_run(input, output)
      → put input/output into those generated structures
      → call tvmgen_default_run(...)
```

So the application calls stable functions such as `app_model_run()`, while the generated adapter handles package specific names. This is model-agnostic at **build time**, not at runtime: a different archive still requires a new firmware build and flash.

> **WHY METADATA IS STILL NECESSARY**
> A `.a` archive is compiled machine code, not a self-describing AI file. It cannot safely tell us its intended preprocessing, labels, tensor meaning, or whether it is compatible. The manifest records those facts, and validation rejects broken or stale metadata.

9. How the NPU and interrupt work together

9.1 What TINIElite does

TINIElite is the on-chip neural-processing accelerator. It executes supported graph stages without asking the Cortex-M33 CPU to perform every operation itself. The generated model is hybrid: some graph work is CPU code and supported stages are sent to the NPU. This project does not claim a measured CPU-versus-NPU speedup because an equivalent CPU-only model artifact is absent.

The NPU has small instruction and parameter memories. The generated runtime loads what it needs for a stage, starts the stage, then later loads the next stage's material. It does not keep the full model in the NPU's small local memory at once.

9.2 What happens when an NPU stage finishes

```
Application thread                    NPU hardware
-----------------                     ------------
start generated model stage  ──────►  working
wait on semaphore            │
                             └──────►  stage completes
                                       raises DONE interrupt 41


short interrupt handler:
  clear DONE status
  count completion
  give semaphore

application thread wakes:
  call generated continuation
  start next stage or finish model
```

The interrupt handler is intentionally tiny. It does not run the large, opaque generated model continuation. It only acknowledges hardware and wakes the normal application context. That is safer and keeps interrupt latency predictable.

9.3 Why a semaphore is better than repeatedly checking

Repeatedly checking a status bit is called polling. It wastes CPU attention while no progress is possible. A semaphore lets the application sleep until the interrupt signals it. It also provides a clean 100 ms deadline: if the signal never arrives, the system treats that as a fault.

10. What happens when something goes wrong

10.1 Bad UART data

The CRC, length, type, version, and sequence checks prevent damaged or stale data from being treated as a valid tensor. The parser can resynchronize by searching for the next message-start bytes. UART hardware errors such as break, framing, parity, or overrun are also reported.

10.2 NPU stage never completes

If the semaphore wait reaches 100 ms without a completion interrupt, the firmware returns a timeout error, gives the UART transmitter a short chance to finish the error message, then performs a cold reset. A full reset is deliberate: the binary model runtime may have hidden partially updated state, and the project does not have its source-level recovery contract.

10.3 Entire application stops progressing

A software timeout cannot help if the main application itself is frozen. The independent hardware watchdog solves that wider problem. It has a four-second period and is fed only after the main loop makes real progress. If the main loop becomes stuck, it stops feeding the watchdog; the watchdog then resets the board.

There is intentionally no background task whose only job is feeding the watchdog. Such a task could keep the board alive while the useful application was deadlocked, defeating the point of the watchdog.

10.4 How the project tests recovery

Diagnostic-only protocol messages can intentionally corrupt a frame, wait for an NPU event without starting the NPU, or withhold watchdog feeding. After a reset, the PC requires a fresh valid READY message and then reruns normal inference. These dangerous commands are controlled by build-time Kconfig options and should not be enabled in a deployed configuration.

11. Where to look in the repository

If you want to understand...	Start here	Why
The main request loop	`app/src/main.c`	It receives requests, dispatches work, sends replies, and feeds the watchdog.
The protocol	`app/src/app_uart.c` and `docs/PROTOCOL.md`	These define frame parsing, CRC, errors, and responses.
NPU waiting and timing	`app/src/app_npu.c`	It starts the model bridge, waits for completion, handles timeout, and records metrics.
Clock correctness	`soc/ti/am13x/clock.c` and `app/src/app_clock.c`	One configures early clocks; one reads them back for verification.
UART hardware setup	`drivers/serial/uart_am13e230x.c`	It controls the UNICOMM UART peripheral.
NPU hardware/interrupt	`drivers/misc/tinyengine/tinyengine_am13e230x.c`	It handles power, NPU memory writes, IRQ 41, counters, and semaphore signalling.
Model selection	`models/<name>/model.json` and `app/CMakeLists.txt`	The manifest states the model contract; CMake validates it and generates the adapter.
PC test client	`tools/send_tensor.py`	It prepares tensors, communicates over serial, and validates results.
Exact evidence and limits	`docs/VALIDATION.md` and `docs/QUALITY_GATES.md`	They distinguish demonstrated results from pending work.

12. What to say in an interview

What did you actually implement?

I integrated supplied TI-generated model archives into an out-of-tree Zephyr port for AM13E230. I added the board/SoC support, clock and pin setup, UART protocol, NPU driver and interrupt handling, model-package selection, fault recovery, host test tooling, and validation workflow. I did not claim to train the original model.

Why use an interrupt instead of checking the NPU in a loop?

The NPU tells the CPU when a stage is complete. The application can wait on a semaphore instead of wasting CPU time polling a register. The design also gives each stage a clear timeout boundary.

What is the model adapter?

It is a tiny C bridge generated at build time from the model manifest. It hides vendor-specific C names and structures behind the stable `app_model.h` interface, so compatible models do not need hand-written integration code.

How do you know the current code is not just "building by accident"?

The project validates model hashes and manifests, runs host and native tests, static checks, and pristine Arm cross-builds. The final ELF is inspected for required model/NPU symbols. Both real protocol-v3 packages were then freshly flashed and passed exact fixtures, clock, CRC, interrupt, NPU-timeout, watchdog, and 600-inference hardware gates.

> **FINAL MENTAL MODEL**
>
> The PC safely sends a correctly shaped tensor. The Zephyr application checks it, passes it through a generated model bridge, waits safely for NPU stages, recovers from failures, and returns both the result and evidence that the accelerator completed its work.

Read Volume 3 next for the project history, validation evidence, and limitations. Return to this volume whenever a term in the source code feels unfamiliar.